

Documentação do Código do Sistema Web

1. Config

O pacote ``Config`` contém classes responsáveis por definir configurações globais da aplicação. Essas configurações incluem segurança, permissões de acesso, controle de sessão e configurações de CORS (Cross-Origin Resource Sharing).

1. 1 APIConfig

A classe ``APIConfig`` gerencia a configuração de segurança da API, incluindo permissões de acesso e configurações de CORS. Ela utiliza anotações do Spring Security para aplicar as regras de segurança de forma centralizada.

=====

2. Controller

O pacote ``Controller`` é responsável por gerenciar as requisições HTTP da API, lidando com a entrada e saída de dados e garantindo que os serviços sejam corretamente acessados. Ele segue o padrão MVC, sendo a camada que recebe as requisições do usuário e interage com o serviço para retornar as respostas adequadas.

2.1 CepController

A classe ``CepController`` é o ponto de entrada para todas as operações relacionadas a CEPs dentro da API. Ela expõe endpoints REST para inserção, listagem e busca de CEPs.

2.1.1 Método `inserirCep`

Recebe um ``CepRequestDTO`` no corpo da requisição e chama o serviço para cadastrar um novo CEP na base de dados. Retorna um objeto ``ApiResponse`` indicando o sucesso da operação.

2.1.2. Método `listarCep`

Lista os CEPs cadastrados com suporte a paginação e ordenação. Os parâmetros opcionais ``cep``, ``sortField`` e ``sortDirection`` permitem filtrar e ordenar os resultados.

2.1.3. Método `buscarCep`

Busca um CEP específico na base de dados através de um parâmetro ``cep`` na URL. Retorna um ``ApiResponse`` contendo os detalhes do CEP encontrado.

3. GlobalExceptionHandler

A classe ``GlobalExceptionHandler`` é responsável pelo tratamento centralizado de exceções dentro da aplicação. Ela intercepta exceções lançadas durante a execução das requisições e retorna respostas padronizadas, melhorando a experiência do usuário e garantindo que erros sejam tratados corretamente.

=====

4. Domain

4.1 Exception

O pacote ``Exception``, localizado dentro de ``Domain``, contém classes de exceção personalizadas utilizadas para lidar com erros específicos relacionados à manipulação de CEPs na API. Essas exceções são lançadas quando ocorrem condições anômalas durante a execução do sistema, garantindo um tratamento adequado dos erros.

4.2 Model

O pacote ``Model``, localizado dentro de ``Domain``, contém as classes que representam as entidades do sistema. Essas classes são responsáveis por mapear os objetos para o banco de dados, seguindo o padrão JPA (Java Persistence API). Cada classe dentro deste pacote define uma estrutura de dados específica utilizada pela API.

4.2.1. Classe Cep

A classe ``Cep`` representa a entidade de um endereço cadastrado no sistema. Ela contém atributos que armazenam informações detalhadas do endereço, como logradouro, bairro, cidade, estado e códigos auxiliares. Essa classe está anotada com ``@Entity`` e ``@Table(name="cep")``, indicando que seus dados são persistidos em uma tabela do banco de dados.

4.2.2. Classe DefaultEntity

A classe ``DefaultEntity`` é uma classe base utilizada para fornecer atributos e comportamentos comuns a outras entidades do sistema. Ela está anotada com ``@MappedSuperclass``, o que significa que seus atributos serão herdados por outras classes, mas sem gerar uma tabela própria no banco de dados.

4.3. Request

O pacote ``Request``, localizado dentro de ``Domain``, contém classes responsáveis por definir a estrutura dos dados que a API recebe nas requisições HTTP. Essas classes ajudam a garantir que as entradas do usuário sejam validadas corretamente antes de serem processadas pelo sistema.

4.3.1. Classe CepViaCepRequestDTO

A classe ``CepViaCepRequestDTO`` é utilizada para representar os dados retornados pela API externa ViaCep. Ela contém atributos detalhados de um endereço que podem ser utilizados internamente pelo sistema.

4.3.2. Classe CepRequestDTO

A classe `CepRequestDTO` é um `Java Record` utilizado para receber e validar os dados enviados pelo cliente ao cadastrar um novo CEP no sistema. Ela aplica anotações de validação para garantir que os campos obrigatórios sejam preenchidos corretamente.

****Validações Implementadas:****

- `@NotBlank(message = "Cep deve ser informado!")`: Garante que o campo `cep` não seja vazio.
 - `@NotBlank(message = "Localidade deve ser informada!")`: Obriga o preenchimento da localidade.
 - `@NotBlank(message = "UF deve ser informada!")`: Exige a presença da Unidade Federativa.
 - `@NotBlank(message = "Estado deve ser informado!")`: Garante que o estado seja informado.
 - `@NotBlank(message = "Região deve ser informada!")`: Torna obrigatório o campo de região.
-

4.4. Response

O pacote `Response`, localizado dentro de `Domain`, contém classes responsáveis por definir a estrutura dos dados retornados pela API nas respostas HTTP. Essas classes garantem que as informações enviadas ao cliente sejam formatadas corretamente, facilitando o consumo da API.

4.4.1. Classe ApiResponse

A classe `ApiResponse` representa a estrutura básica de resposta da API. Ela contém uma mensagem (`message`) indicando o status da operação e um objeto (`data`) que pode conter qualquer tipo de dado retornado.

4.4.2. Classe CepResponseDTO

A classe `CepResponseDTO` é utilizada para representar a resposta da API quando um CEP é recuperado do banco de dados. Ela encapsula todas as informações de um endereço relacionadas ao CEP pesquisado.

4.4.3 Classe CepSearchedResponseDTO

A classe `CepSearchedResponseDTO` é utilizada quando um CEP é pesquisado diretamente na API. Ela contém os mesmos atributos de `CepResponseDTO`, mas sem um identificador (`id`), pois os dados podem vir de uma fonte externa.

4.4.4. Classe PageableResponse

A classe `PageableResponse<T>` é responsável por encapsular a resposta paginada da API. Ela utiliza a estrutura de `Page<T>` do Spring Data para organizar os dados e retornar informações essenciais sobre a paginação.

****Principais Atributos:****

- `content`: Lista de objetos do tipo `T` representando os dados retornados na página atual.
- `totalItems`: Número total de registros disponíveis.
- `currentPage`: Número da página atual.
- `lastPage`: Número total de páginas disponíveis.
- `perPage`: Quantidade de itens por página.

4.5. Validation

O pacote ``Validation``, localizado dentro de ``Domain``, contém classes responsáveis por validar os dados relacionados aos CEPs antes de serem processados pelo sistema. Ele garante que as regras de negócio sejam aplicadas antes da persistência no banco de dados, evitando erros e inconsistências.

4.5.1. Classe `CepValidation`

A classe ``CepValidation`` é responsável por validar se um CEP é válido e se já está cadastrado no sistema. Ela é anotada com ``@Service``, permitindo que seja gerenciada pelo Spring IoC (Inversion of Control).

4.5.1.1 Método `isValidCep`

O método `isValidCep(String cep)`` verifica se o CEP informado é válido. Ele realiza duas validações principais:

- Se o valor do CEP está vazio, lançando uma exceção ``InvalidCepException``.
 - Se o CEP não possui exatamente 8 dígitos numéricos, lançando outra ``InvalidCepException``.
- Isso garante que apenas CEPs válidos sejam processados pelo sistema.

4.5.1.2. Método `isAlreadySaved`

O método `isAlreadySaved(String cep)`` verifica se um CEP já está cadastrado no banco de dados antes de inseri-lo. Caso o CEP já exista, uma exceção ``AlreadyRegisteredCepException`` é lançada, evitando duplicações.

5. HTTP

O pacote ``Http`` contém classes responsáveis por realizar a comunicação externa da API. Nesse contexto, ele implementa a integração com a API pública ViaCep para obtenção de informações de endereços a partir de um CEP informado.

5.1. Interface `ViaCepClient`

A interface ``ViaCepClient`` define um cliente HTTP para comunicação com a API ViaCep, utilizando o Spring Cloud OpenFeign para simplificar requisições externas.

5.1.1. Anotação `@FeignClient`

A anotação `@FeignClient(name = "ViaCepApi", url = "https://viacep.com.br/ws")`` define a configuração do cliente Feign. Ela especifica o nome do cliente (``ViaCepApi``) e a URL base para as requisições.

5.1.2. Método `buscaDadosDoCep`

O método `buscaDadosDoCep(@PathVariable String cep)`` realiza uma requisição HTTP GET para o endpoint ``/{cep}/json`` da API ViaCep. Ele recebe um CEP como parâmetro e retorna um objeto ``CepViaCepRequestDTO`` contendo as informações do endereço correspondente.

6. Repository

O pacote ``Repository`` contém interfaces responsáveis pela persistência de dados no banco de dados, seguindo o padrão **Spring Data JPA**. Ele abstrai operações de banco, permitindo consultas otimizadas e de fácil manutenção.

6.1. Interface `CepRepository`

A interface ``CepRepository`` é a camada responsável pela comunicação com o banco de dados para operações relacionadas à entidade ``Cep``. Ela herda de ``JpaRepository<Cep, Long>``, fornecendo métodos básicos de persistência, além de consultas personalizadas.

6.1.1. Método `findByAllByParam`

Realiza uma busca de registros na tabela de CEPs com suporte a **filtros dinâmicos e ordenação personalizada**. Os parâmetros aceitos são:

- ``cep``: Filtro opcional para buscar CEPs específicos.
- ``sortField``: Define o campo utilizado para ordenação (exemplo: ``localidade``, ``bairro``, ``estado``).
- ``sortDirection``: Direção da ordenação (``asc`` para crescente, ``desc`` para decrescente).
- ``Pageable``: Objeto que permite a paginação e ordenação dos resultados.

O método utiliza uma **query nativa SQL** para ordenar dinamicamente os resultados com base no campo especificado.

6.1.2. Método `existsByCep`

Verifica se um determinado CEP já existe na base de dados, retornando ``true`` caso o registro já esteja cadastrado e ``false`` caso contrário. Esse método é utilizado para impedir a duplicação de registros.

6.1.3. Método `findByCep`

Busca um CEP específico na base de dados e retorna um ``Optional<Cep>``. Se o CEP existir, o objeto ``Cep`` é retornado; caso contrário, o resultado será ``Optional.empty()``.

=====

7. Service

O pacote ``Service`` contém classes responsáveis pela lógica de negócios da aplicação. Ele atua como intermediário entre os controladores (Controller) e a camada de persistência (Repository), garantindo que os dados sejam processados corretamente antes de serem armazenados ou retornados.

7.1. Classe `CepService`

A classe ``CepService`` gerencia as operações relacionadas aos registros de CEP no sistema. Ela fornece métodos para cadastrar, buscar e listar CEPs, além de integrar com a API externa ViaCep.

7.1.1. Método `registerAddress`

Registra um novo CEP no banco de dados. Antes de salvar, verifica se o CEP já está cadastrado, construindo a entidade ``Cep`` e persistindo-a no repositório.

7.1.2. Método `getAddresses`

Retorna uma lista paginada de CEPs com suporte a filtros e ordenação. Os parâmetros ``sortField`` e ``sortDirection`` permitem ordenar os registros por ``localidade``, ``bairro`` ou ``estado``, em ordem crescente ou decrescente.

7.1.3. Método getAddress

Busca um CEP no banco de dados. Caso não encontre, consulta a API externa ViaCep para obter os dados e retorna um objeto `Cep` com as informações do endereço.

7.1.4. Método isRegistered

Verifica se um CEP já está registrado no banco de dados, retornando um valor booleano (`true` se encontrado, `false` caso contrário).

7.1.5. Método requestViaCep

Realiza uma requisição à API ViaCep para buscar um endereço com base no CEP fornecido. Se a resposta for nula ou inválida, lança uma exceção `InvalidCepException`.

7.1.6. Método buildAddressShow

Constrói um objeto `Cep` a partir da resposta da API ViaCep, preenchendo os dados do endereço obtido.

7.1.7. Método buildAddressToInsert

Constrói um objeto `Cep` para ser inserido no banco de dados com base nos dados fornecidos pelo usuário na requisição.

